

MeetingIQ

Your AI-Powered Meeting Intelligence Platform

Project Overview

MeetingIQ is a powerful, intelligent, and elegantly designed AI-powered meeting intelligence web application built to transform raw meeting recordings into structured, actionable business intelligence. Built using FastAPI as the backend framework, Vanilla JavaScript and HTML5/CSS3 for the frontend, Groq's Whisper large-v3 model for speech-to-text transcription, Groq's LLaMA 3.3 70B model as the primary analysis engine, and Google Gemini 1.5 Flash as an intelligent fallback, the platform delivers a fast, reliable, and deeply insightful experience that goes far beyond typical transcription tools.

MeetingIQ is centred around an automated intelligence pipeline that takes any audio or video recording of a meeting and produces a comprehensive structured output including an executive summary, prioritised action items with assignees and deadlines, key decisions with their impact statements, participant identification, key insights, next steps, overall meeting effectiveness ratings, and sentiment analysis. The application processes the audio with industry-leading accuracy using Groq's Whisper large-v3 transcription model and then passes the resulting transcript to either LLaMA 3.3 70B or Gemini 1.5 Flash for deep language understanding and structured extraction.

Beyond the core AI processing pipeline, MeetingIQ offers a rich and polished user experience featuring a glassmorphic dark-mode interface, a drag-and-drop file upload zone that accepts virtually every common audio and video format, animated loading status updates, an interactive results dashboard with a stats overview, a colour-coded action items list with checkbox tracking, a key decisions panel with impact statements, a participant roster, full transcript viewing, and a one-click reset to process another recording. The visual design uses a deep navy background with purple and cyan gradient accents, creating a modern and professional aesthetic that reflects the intelligence of the platform itself.

The application is built on a lean, purposeful architecture. The FastAPI backend is lightweight and exposes only the API endpoints required for recording processing and health monitoring, while the entire frontend runs as a zero-framework single-page application with no React, Vue, or Angular required. The system handles large recordings gracefully through automatic audio extraction from video files using MoviePy and intelligent chunked transcription that splits recordings longer than twenty-two megabytes into five-minute segments, transcribes each independently, and joins the results seamlessly.

Through the thoughtful integration of state-of-the-art speech recognition, large language model analysis, and a robust dual-AI fallback architecture, MeetingIQ delivers a meeting intelligence platform that is fast, accurate, resilient, and genuinely useful for professionals of all technical backgrounds who need to extract value from recorded conversations.

Scenario 1: AI-Powered Meeting Transcription and Intelligence Extraction

Business professionals sit through hundreds of meetings every year. Yet the value created in those meetings — the decisions made, the tasks assigned, the insights surfaced — is consistently lost, buried in incomplete notes or simply forgotten within hours. Traditional note-taking is slow, error-prone, and deeply distracting. Recording software captures audio but leaves all the cognitive work of extraction to the human listener. Even dedicated transcription services produce raw text that still requires significant effort to convert into actionable deliverables.

MeetingIQ addresses this challenge directly by automating the entire intelligence extraction pipeline from upload to structured output. A user can drag any recording — a Zoom call, a Teams meeting, a voice memo, a video interview — onto the upload zone, optionally provide a meeting title, and click Process Recording. Within minutes, the application returns a complete intelligence report that a human would have taken hours to produce manually.

The pipeline begins with audio extraction. For video files, MoviePy silently strips the audio track and writes a compressed sixty-four kilobit MP3 to a temporary session directory. For audio files, MoviePy handles format conversion to the same standardised MP3 output. This normalisation step ensures that the Groq Whisper API receives a clean, consistent input regardless of whether the original recording was an MP4 screen capture, an MKV video, an OGG voice note, or a WAV audio file.

Transcription is handled by Groq's Whisper large-v3 model, which provides state-of-the-art multilingual speech recognition with exceptional accuracy across a wide range of speakers, accents, and recording conditions. For files under the twenty-two megabyte threshold, the entire audio is sent in a single API call with the `response_format` set to text for immediate string extraction. For larger files — long meetings, all-day workshops, extended interviews — the system automatically switches to a chunked transcription mode that splits the audio into five-minute segments using MoviePy's `subclip` functionality, transcribes each segment independently, and joins all results with spacing to produce a seamless continuous transcript.

Once the transcript is available, MeetingIQ passes it to the analysis layer. For transcripts under forty thousand characters, Groq's LLaMA 3.3 70B Versatile model performs the analysis with temperature set to 0.3 for consistent and precise structured output. For very long transcripts, Google Gemini 1.5 Flash handles the analysis instead, providing the same structured JSON output through a different inference pathway. Both models receive the same detailed system prompt that defines the exact JSON schema for the response and specifies comprehensive rules for extraction quality.

The analysis output is a structured JSON object containing the meeting title, a two to three sentence executive summary, an array of key topics, a full action items list with task descriptions, assignees, priority ratings, deadline suggestions, category tags, and status fields, a decisions array with impact statements and decision-makers, key insights, an overall next steps paragraph, a meeting effectiveness rating, a sentiment assessment, a follow-up required flag, and a participant list. This structured intelligence is then rendered beautifully in the results dashboard, where users can interact with their action items, review decisions, read the full transcript, and immediately understand the value and outcomes of their meeting.

Scenario 2: Large File Handling and Resilient Processing

Meeting recordings in real-world professional environments vary enormously in length, format, and file size. A quick fifteen-minute standup might be a twelve-megabyte MP3, while a quarterly all-hands or a full-day workshop could easily be a two-gigabyte MP4 or a multi-hour WAV file.

Any meeting intelligence platform that claims to be genuinely useful must handle this full range of inputs gracefully, without requiring users to pre-process their files, without silently truncating content, and without exposing cryptic API errors when file sizes exceed thresholds.

MeetingIQ addresses this challenge through a two-stage resilience architecture built around automatic format normalisation and transparent size-based routing. The first stage handles format diversity. Rather than accepting only audio files or requiring users to convert their recordings before uploading, MeetingIQ accepts eighteen distinct file formats including MP4, MKV, MOV, AVI, WMV, FLV, WebM, M4V, 3GP, and Transport Stream formats on the video side, and MP3, WAV, M4A, AAC, OGG, FLAC, WMA, and Opus on the audio side. Every uploaded file is immediately processed by MoviePy's `extract_audio` function, which handles both `VideoFileClip` and `AudioFileClip` paths to produce a standardised compressed MP3 regardless of source format.

The second stage handles size diversity. The Groq Whisper API imposes an approximately twenty-five megabyte per request limit. MeetingIQ applies a conservative twenty-two megabyte threshold and routes files below this limit through the direct single-request transcription path. Files above the threshold, or files that fail the single-request path with a size-related error, are automatically routed to the chunked transcription path without any user intervention or error messaging. The chunked path uses MoviePy's `subclip` function to extract sequential five-minute segments from the full audio, writes each segment to a temporary chunk file at sixty-four kilobit compression, transcribes it independently through the Groq API, appends the result to a running transcript array, and cleans up the chunk file before proceeding to the next segment. The completed array is joined with spaces to produce the final continuous transcript.

This architecture means that the user experience is entirely uniform regardless of file size. A two-minute recording and a three-hour recording both produce the same results dashboard with the same structured intelligence output. The only difference visible to the user is processing time, which the animated loading overlay communicates through four sequential status messages Uploading, Extracting audio, Transcribing speech, and Analyzing insights — that reflect the actual pipeline stages in progress.

Key Features and Benefits

AI Transcription Engine Groq Whisper large-v3

The transcription core of MeetingIQ powered by Groq's deployment of OpenAI's Whisper large-v3 model. This state-of-the-art speech recognition system delivers exceptional accuracy across a wide range of speakers, accents, technical vocabularies, and recording conditions. Groq's LPU inference hardware provides significantly faster-than-real-time transcription speeds, meaning a sixty-minute meeting recording typically transcribes in under two minutes. The system handles both direct single-request transcription for standard files and fully automatic chunked transcription for large files, with all routing handled transparently in the backend.

Structured Intelligence Extraction LLaMA 3.3 70B Primary

Following transcription, MeetingIQ applies a comprehensive AI analysis pipeline powered by Groq's LLaMA 3.3 70B Versatile model as the primary engine. The model receives the full transcript alongside a carefully engineered system prompt that defines an exact twelve-field JSON schema and provides detailed instructions for extraction quality, completeness, and formatting. The analysis runs at temperature 0.3 to prioritise consistency and structural accuracy over creative variation. The model extracts all action items including implied ones, identifies

decision-makers, suggests realistic deadlines based on contextual language cues, rates meeting effectiveness and sentiment, and auto-detects participants by name from the transcript.

Google Gemini 1.5 Flash Fallback

For very long transcripts exceeding forty thousand characters or in cases where the primary Groq analysis path fails, MeetingIQ automatically falls back to Google Gemini 1.5 Flash for analysis. The same structured JSON schema and system prompt are applied to the Gemini API call, ensuring that the output format is identical regardless of which model performed the analysis. This dual-engine architecture provides meaningful resilience against rate limits, service disruptions, and edge cases in transcript length, ensuring that users always receive their intelligence report even when one AI service is under pressure.

Interactive Results Dashboard

The results section of MeetingIQ is a carefully designed single-page dashboard that renders all extracted intelligence in a clear, scannable, and interactive layout. The top stats row displays meeting effectiveness, sentiment, and total action item count at a glance. The main content area uses a two-column grid on desktop with the executive summary, the full action items list with interactive checkboxes, and the complete transcript in the left column, and the key decisions panel, participant roster, and next steps paragraph in the right column. The dashboard includes a back button to process a new recording and animates into view with a smooth fade-in transition.

Colour-Coded Action Items with Priority and Deadline Tracking

The action items section is the operational heart of the results dashboard. Each action item is rendered as an interactive card with a checkbox for completion tracking, a priority badge colour-coded as red for High, amber for Medium, and green for Low, a category label drawn from a taxonomy covering Development, Design, Testing, Communication, Planning, Research, Finance, HR, and Other, the full task description, the identified or inferred assignee, and the suggested deadline. Items are presented in the order extracted by the AI, which naturally surfaces blocking and urgent items first given the system prompt's instructions to treat urgency and blocking status as High priority signals.

Key Decisions Panel with Impact Statements

The decisions panel captures the strategic outcomes of the meeting separately from the tactical action items. Each decision is rendered as a card with a distinctive cyan left border, showing the decision statement and its impact description. The impact field provides the contextual consequence of the decision — why it matters, what it unblocks, or what it changes — giving reviewers who were not in the meeting immediate understanding of the significance of each outcome. The `made_by` field identifies whether the decision was made by the group collectively or by a named individual.

Broad Format Support and Automatic Audio Extraction

MeetingIQ accepts recordings in eighteen distinct audio and video formats including MP4, MKV, MOV, AVI, WMV, FLV, WebM, M4V, 3GP, TS, MTS, M2TS on the video side and MP3, WAV, M4A, AAC, OGG, FLAC, WMA, Opus on the audio side. All files are normalised to a compressed MP3 using MoviePy before transcription, eliminating the need for any user-side format conversion. This means MeetingIQ works directly with screen recordings from Zoom or Teams, voice memos from smartphones, dedicated audio recorders, podcast exports, and any other recording source that produces a standard media file.

Drag-and-Drop Upload Interface

The upload interface is built around a large, visually prominent drag-and-drop zone that accepts files via both dragging and conventional file picker interaction. The zone provides immediate visual feedback on dragover with a subtle border glow and upward transform animation. On file selection, the file name and size are displayed immediately below the drop zone, confirming the selection without requiring the user to navigate away. The Process Recording button activates only after a valid file has been selected, preventing empty submissions.

Animated Processing Feedback

During the processing pipeline, a full-screen loading overlay with a backdrop filter blur effect creates a focused waiting experience. A spinning gradient ring animation provides clear visual confirmation that processing is underway. Four sequential status messages — Uploading, Extracting audio, Transcribing speech, and Analyzing insights — update every five seconds on a timer to reflect the actual pipeline stages, setting accurate expectations about processing time and keeping the user informed throughout what can be a multi-minute operation for longer recordings.

Full Transcript Access

The complete meeting transcript generated by Whisper is displayed in a scrollable pre-formatted panel within the results dashboard. Users can review the raw transcript to verify transcription accuracy, search for specific passages, or copy portions for use in other documents. Displaying the full transcript alongside the AI-extracted intelligence serves an important trust and verification function — users can always trace any action item or decision back to the original words that generated it, maintaining full transparency in the AI analysis process.

Architecture Overview

MeetingIQ follows a clean, modular client-server architecture consisting of three clearly defined layers, each responsible for a distinct part of the application's functionality. This separation of concerns ensures that the system is easy to maintain, straightforward to extend, and reliable under normal usage conditions.

The three layers are: Presentation Layer (Browser-Side Single-Page Application), Application Logic Layer (FastAPI Backend API), and AI Services Layer (Groq Whisper, Groq LLaMA, and Google Gemini). Each layer communicates with the next through well-defined interfaces — the frontend communicates with the FastAPI backend via JSON-based Fetch API calls using multipart form data for file uploads, and the FastAPI backend communicates with the external AI services via their respective Python SDKs and HTTP clients.

Presentation Layer (Frontend SPA — index.html)

The entire frontend of MeetingIQ lives in a single HTML file that functions as a full single-page application. Built with carefully structured Vanilla JavaScript, HTML5, and CSS3, the application uses CSS custom properties to define its complete visual design system including the dark navy background, glassmorphic surface treatment, purple primary gradient, cyan accent, and all typography scales. Navigation between the upload section and results section is handled by direct JavaScript DOM manipulation that shows and hides the two main content areas, giving users a smooth, app-like experience without any page reload.

The frontend is responsible for all user interaction including drag-and-drop file handling, form field management, file selection feedback, processing state management, Fetch API calls to the backend with multipart form data, JSON response parsing, dynamic DOM rendering of all results panels, interactive checkbox state for action items, and the reset flow that returns users to the upload interface.

Application Logic Layer (FastAPI Backend main.py)

The FastAPI backend is a concise and focused REST API server. It does not serve a database, does not manage user sessions on the server side, and does not persist any uploaded content beyond the duration of a single processing request. Its sole responsibilities are to serve the frontend HTML file, receive multipart form data uploads from the browser, extract and normalise audio using MoviePy, route to the appropriate transcription path based on file size, call the Groq Whisper API for transcription, call the analysis AI service for intelligence extraction, parse and validate the JSON response, and return the structured intelligence payload to the frontend.

The backend exposes three endpoints: POST /api/process for full recording analysis, POST /api/analyze-text for direct transcript text analysis, and GET /health for system status monitoring. All endpoints include comprehensive error handling with user-friendly HTTP exception messages, ensuring the frontend always receives a meaningful error response rather than a Python traceback when something goes wrong.

AI Services Layer (Groq + Google Gemini)

Three external AI services power the intelligent features of MeetingIQ. Groq provides speech-to-text intelligence through the Whisper large-v3 model and primary text analysis intelligence through the LLaMA 3.3 70B Versatile model. Both Groq integrations use the official Groq Python SDK, which provides a clean and familiar OpenAI-compatible interface. Google Gemini provides secondary analysis intelligence through the Gemini 1.5 Flash model, accessed via the google-generativeai Python SDK. All three services are configured via environment variables stored in a .env file that is never committed to version control.

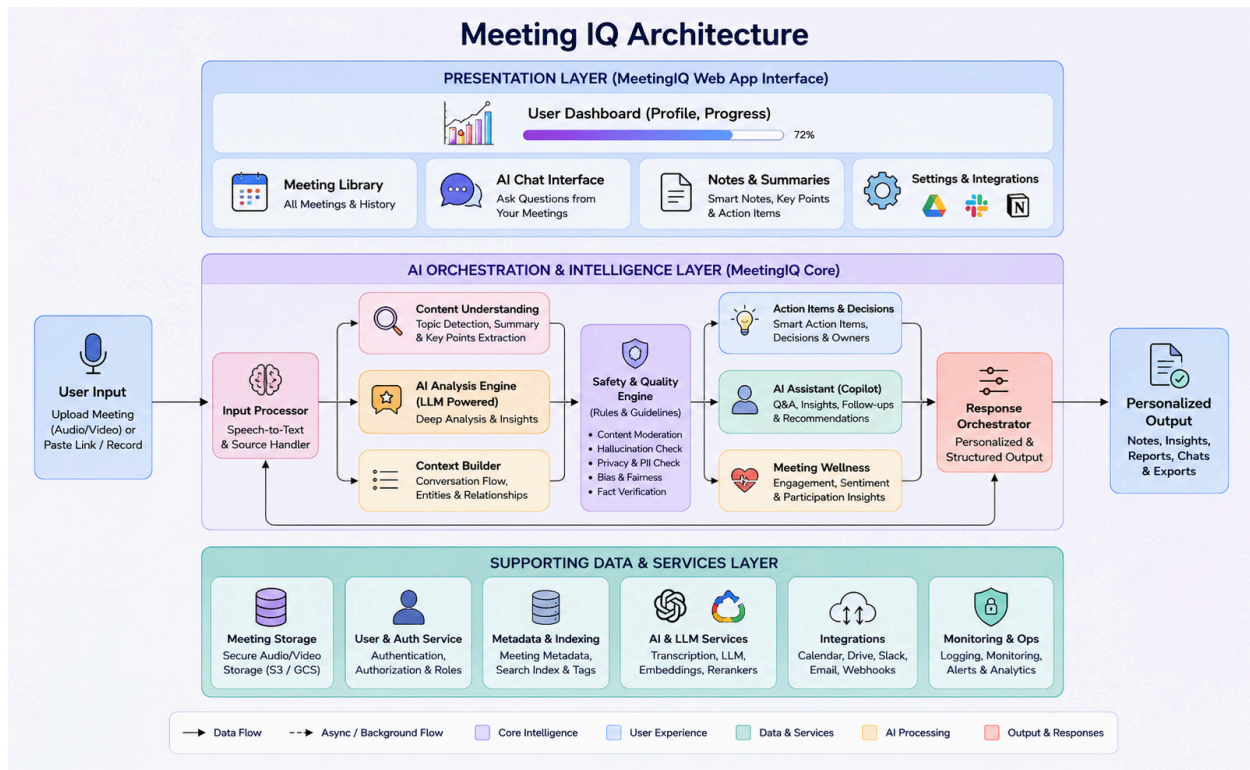


Figure 1: Architecture Overview

Core Technologies

MeetingIQ is built using a carefully selected stack of modern web and AI technologies. Every component has been chosen for a specific purpose, with a deliberate emphasis on keeping dependencies minimal, performance high, and the development experience straightforward.

FastAPI (Backend Framework)

FastAPI is a modern, high-performance Python web framework built on top of Starlette and Pydantic. It serves as the backbone of MeetingIQ's server-side logic, handling HTTP routing, multipart file upload parsing, JSON response serialisation, and template file serving. FastAPI's async-first design means that file I/O operations and API calls do not block the server thread, which is important given the multi-second duration of transcription and analysis operations. FastAPI also automatically generates OpenAPI documentation from route definitions, providing a built-in API reference at /docs for development and debugging. Uvicorn serves as the ASGI server that runs the FastAPI application.

Groq API Whisper large-v3 (Transcription Engine)

Groq provides cloud-based inference for OpenAI's Whisper large-v3 speech recognition model, which powers MeetingIQ's transcription capability. Whisper large-v3 is the current state-of-the-art in open-source automatic speech recognition, delivering exceptional accuracy across diverse speakers, accents, languages, and recording conditions. Groq's proprietary LPU (Language Processing Unit) inference hardware provides transcription at significantly faster-than-real-time speeds, typically processing a sixty-minute meeting in under two minutes. The Groq Python SDK provides a clean interface that accepts binary audio data directly without requiring temporary file uploads to external storage.

Groq API LLaMA 3.3 70B Versatile (Primary Analysis Engine)

Groq also provides cloud-based inference for Meta's LLaMA 3.3 70B Versatile model, which powers MeetingIQ's primary intelligence extraction capability. LLaMA 3.3 70B is a state-of-the-art large language model that delivers high-quality structured text generation with excellent instruction following and JSON schema adherence. The model is called with temperature 0.3 and a four thousand token maximum to produce consistent, well-structured analysis outputs. Groq's LPU hardware provides low-latency inference even for large models, typically returning a complete analysis of a one-hour meeting transcript in under thirty seconds.

Google Gemini API gemini-1.5-flash (Fallback Analysis Engine)

Google's Gemini 1.5 Flash model serves as the fallback analysis engine for MeetingIQ, activated automatically for very long transcripts exceeding forty thousand characters or when the primary Groq analysis path encounters an error. Gemini 1.5 Flash offers a very large context window that accommodates even the longest meeting transcripts without truncation, and its fast inference speed makes it practical as a production fallback rather than an emergency measure. The model is accessed via the google-generativeai Python SDK and receives the same system prompt and schema requirements as the primary Groq LLaMA call.

```
35  gemini_key = os.environ.get("GEMINI_API_KEY")
36  if gemini_key:
37      genai.configure(api_key=gemini_key)
38      gemini_model = genai.GenerativeModel('gemini-1.5-flash')
39  else:
40      print("WARNING: GEMINI_API_KEY not found. Fallback disabled.")
41      gemini_model = None
```

Figure 2: Gemini Logic

MoviePy (Audio Extraction and Processing)

MoviePy is the Python library that handles all audio extraction and processing in MeetingIQ. It provides a unified interface for working with both VideoFileClip and AudioFileClip objects, enabling the backend to extract audio from video files and normalise audio from any source format into a compressed MP3 at sixty-four kilobit bitrate. MoviePy also powers the chunked transcription pipeline, where its subclip function extracts precise time segments from the full audio clip for independent transcription. MoviePy is built on top of FFmpeg and supports virtually every common audio and video codec.

Python 3.9+ (Core Backend Language)

Python serves as the foundation of the entire backend, enabling seamless integration of FastAPI routing, the Groq SDK, the Google Generative AI SDK, MoviePy audio processing, UUID session management, temporary directory handling, JSON serialisation, and comprehensive error handling. Python's readability and rich ecosystem make it the natural choice for AI-integrated web applications, and its broad deployment support across all major cloud platforms ensures MeetingIQ can be hosted almost anywhere.

HTML5 / CSS3 / Vanilla JavaScript (Frontend SPA)

The entire frontend is a single HTML file with no framework dependencies, no build tools, and no npm. This was a deliberate design decision to maximise simplicity, performance, and portability. HTML5 provides the structural foundation with semantic section elements, CSS3 with

custom properties delivers a complete glassmorphic design system with the dark navy background, blur effects, gradient borders, and smooth animations, and Vanilla JavaScript handles all file handling, drag-and-drop events, Fetch API calls, loading state management, and dynamic DOM rendering. The result is a fast-loading, lightweight application that works in any modern browser without installation.

```

312 @app.get("/")
313 async def serve_frontend():
314     return FileResponse(str(Path(__file__).parent / "index.html"))
315 if __name__ == "__main__":
316     import uvicorn
317     uvicorn.run(app, host="127.0.0.1", port=8000)
318

```

Figure 3: Frontend Route

python-dotenv (Environment Variable Management)

The python-dotenv library loads environment variables from a .env file into the application's runtime environment at startup. This is the standard approach for keeping sensitive credentials like API keys out of source code. The .env file stores the GROQ_API_KEY and GEMINI_API_KEY values and should always be listed in .gitignore to prevent accidental exposure in version control systems.

Google Fonts Outfit and Inter (Typography)

Outfit is a modern geometric sans-serif typeface used for the MeetingIQ brand name and primary headings, giving the application its clean and technically confident character. Inter is a highly legible humanist sans-serif used for all body text, labels, and UI copy, ensuring comfortable reading of long transcripts and dense intelligence reports. Both fonts are loaded asynchronously from Google Fonts with rel=preconnect for performance.

```

<style>
:root {
  --primary: #8b5cf6;
  --primary-glow: rgba(139, 92, 246, 0.5);
  --secondary: #ec4899;
  --accent: #06b6d4;
  --background: #0f172a;
  --surface: #1e293b;
  --surface-light: #334155;
  --text: #f8fafc;
  --text-muted: #94a3b8;
  --success: #10b981;
  --warning: #f59e0b;
  --error: #ef4444;
  --glass: rgba(30, 41, 59, 0.7);
  --glass-border: rgba(255, 255, 255, 0.1);
}

```

Figure 4: Styles Illustration

Component-Wise Architecture

Component	Description
Frontend SPA(index.html)	A zero-framework single-page application handling all navigation, file upload, drag-and-drop interaction, processing state management, and results rendering. Built with Vanilla JavaScript, HTML5, and CSS3 with a complete CSS variable dark-mode design system.
FastAPI Backend(main.py)	Lightweight async REST API server exposing three endpoints for recording processing, text analysis, and health monitoring. Handles all audio extraction, size-based routing, transcription orchestration, and analysis API communication.
Audio ExtractionModule	Accepts uploaded files across 18 formats, writes to a temporary session directory, and uses MoviePy's VideoFileClip or AudioFileClip to extract and normalise audio to a compressed 64kbps MP3 for transcription.
ChunkedTranscription Module	Handles recordings that exceed the 22MB direct API threshold by splitting audio into 5-minute segments using MoviePy's subclip function, transcribing each segment independently via Groq Whisper, and joining results into a continuous transcript.
Groq WhisperService	Primary speech-to-text engine. Accepts binary MP3 audio data, returns plain text transcripts with high accuracy across diverse speakers and conditions. Runs on Groq's LPU hardware for fast-than-real-time processing.
Groq LLaMA3.3 70B Service	Primary analysis engine. Receives full transcript with detailed system prompt and JSON schema, returns structured intelligence payload covering action items, decisions, summary, participants, sentiment, and effectiveness at temperature 0.3.
Google Gemini1.5 Flash Service	Fallback analysis engine activated for transcripts over 40,000 characters or on primary engine failure. Receives the same system prompt and schema, returns identical JSON structure via the google-generativeai SDK.
IntelligenceSchema(NOTES_SYSTEM_PROMPT)	The master analysis configuration defining the 12-field JSON response schema and extraction rules. Defines that all action items including implied ones must be extracted, that High priority means urgent or blocking, and that only valid JSON is returned with no markdown fences.
SessionManagement	Each upload creates a unique UUID-based session directory under a system temp path. All intermediate files (input recording, extracted MP3, chunk files) are written here and the entire directory is deleted via shutil.rmtree in the finally block after processing completes.
ResultsDashboard(Frontend)	Renders the complete intelligence payload returned by the API. Populates the stats row, executive summary, action items list with interactive checkboxes and priority badges, decisions panel, participant tags, next steps, and full transcript viewer.
/api/processEndpoint	POST endpoint accepting multipart form data with a file upload and optional title field. Orchestrates the full pipeline: file validation, audio

	extraction, transcription routing, analysis, and JSON response. Cleans up temp files in all exit paths.
/api/analyze-textEndpoint	POST endpoint accepting a JSON body with raw transcript text and optional title. Bypasses the audio pipeline and calls the analysis engine directly, enabling analysis of transcripts from external transcription services or manual text input.
/healthEndpoint	GET endpoint returning a JSON status object with the application name and the names of the active transcription and notes models, useful for deployment health checks and configuration verification.

Intelligence Extraction System Prompt Design

The `NOTES_SYSTEM_PROMPT` constant in `main.py` is the single most important configuration element in MeetingIQ. It defines the exact structured output that both the LLaMA and Gemini analysis engines must produce, the rules they must follow for extraction completeness and quality, and the specific constraints around output format. Designing this prompt carefully is critical because even small changes to its wording can significantly affect the quality, completeness, and consistency of the extracted intelligence.

```

154 NOTES_SYSTEM_PROMPT = """You are an expert meeting analyst and executive assistant AI.
155 Your job is to transform raw meeting transcripts into structured, actionable intelligence.
156
157 Always respond with a valid JSON object following this exact schema:
158 {
159     "title": "Auto-detected or inferred meeting title",
160     "summary": "2-3 sentence executive summary",
161     "key_topics": ["topic1", "topic2", "topic3"],
162     "action_items": [
163         {
164             "id": 1,
165             "task": "Clear, specific task description",
166             "assignee": "Person responsible or 'TBD'",
167             "priority": "High|Medium|Low",
168             "deadline": "Suggested deadline (e.g. 'By Friday', 'Next sprint', 'Within 48h')",
169             "category": "Development|Design|Testing|Communication|Planning|Research|Finance|HR|Other",
170             "status": "Pending"

```

Figure 5: Prompt Illustration

JSON Schema Design

The system prompt defines a strict twelve-field JSON schema that the AI must populate for every analysis. The schema is presented inline in the prompt as a formatted JSON object with field names, type annotations in the form of example values, and nested structure for action items and decisions. The schema includes the meeting title (auto-detected from transcript if not provided), a two to three sentence executive summary, an array of key topics, an action items array with seven sub-fields per item, a decisions array with four sub-fields per item, a key insights array, a next steps paragraph, a meeting effectiveness rating, a sentiment rating, a follow-up required boolean, and a participants array.

Action Item Extraction Rules

The prompt provides specific rules for action item extraction that go beyond simple instruction following. All action items must be extracted including implied ones — if a transcript says 'we should probably get legal to review this before launch', that implies a task even though it was

not stated as a direct instruction. High priority is explicitly defined as urgent or blocking. Medium priority means important but not immediately blocking. Low priority means nice-to-have. The deadline field is instructed to suggest realistic values based on contextual language cues such as 'by Friday', 'before the next sprint', or 'within forty-eight hours' rather than returning generic placeholder values.

Output Format Constraints

The most critical rule in the system prompt is the final instruction: Return ONLY valid JSON, no markdown fences, no extra text. This is essential because language models naturally tend to wrap JSON responses in markdown code fences or precede them with explanatory text. Any such wrapping would cause the JSON parsing in the backend to fail. The backend includes a defensive stripping step that removes markdown fences if they appear, but the prompt instruction minimises their occurrence. Both the LLaMA and Gemini implementations share this prompt, ensuring consistent output formatting across both engines.

Dual-Engine Consistency

A key architectural decision in MeetingIQ is that both the primary and fallback analysis engines receive exactly the same system prompt. This ensures that the results dashboard renders identically regardless of which engine processed a given transcript. The frontend JavaScript does not need to know which engine was used and can treat the JSON payload identically in all cases. The model name is not surfaced to the user in the current interface, though the backend includes it as a field that could be used for transparency reporting in a future version.

Pre-requisites

Before running or deploying MeetingIQ, ensure your development environment is properly configured. The following components and tools are required to run the application and enable all AI-powered features.

1. Python Environment Setup

MeetingIQ is developed using Python 3.9 or higher. It is strongly recommended to use a dedicated virtual environment to isolate the project's dependencies from your system Python installation and from other projects on your machine.

Create and activate a virtual environment using the following commands:

```
python -m venv meetingiq_env
```

```
source meetingiq_env/bin/activate # Linux / macOS
```

```
meetingiq_env\Scripts\activate # Windows
```

Official Python documentation and downloads: <https://www.python.org/downloads/>

2. Install Required Python Packages

Once your virtual environment is active, install all required backend dependencies using pip:

```
pip install fastapi uvicorn[standard] python-dotenv groq python-multipart moviepy  
google-generativeai
```

Alternatively, if a requirements.txt file is present in the project root, install all dependencies at once:

```
pip install -r requirements.txt
```

The requirements.txt specifies exact versions: fastapi==0.115.0, uvicorn[standard]==0.30.6, python-dotenv==1.0.1, groq>=0.11.0, python-multipart==0.0.12, moviepy==1.0.3, and google-generativeai==0.8.3.

3. Obtain and Configure API Keys

MeetingIQ requires two external API keys to enable its AI features. These must be set as environment variables before starting the application.

- Obtain a Groq API key from the Groq developer console: <https://console.groq.com>
- Obtain a Gemini API key from Google AI Studio: <https://aistudio.google.com>

Create a file named .env in the project root directory with the following content:

GROQ_API_KEY=your_groq_api_key_here

GEMINI_API_KEY=your_gemini_api_key_here

```
29  api_key = os.environ.get("GROQ_API_KEY")
30  if not api_key:
31      print("WARNING: GROQ_API_KEY not found in environment or .env file.")
32
33  client = Groq(api_key=api_key)
34
```

Figure 6: GROQ API

Never commit your .env file to version control. Add it to your .gitignore file immediately after creation.

4. Project File Structure

MeetingIQ uses a flat project structure with all files in the project root. Flask-style template directories are not required because FastAPI's FileResponse serves index.html directly from the root.

meetingiq/ ← project root
 main.py ← FastAPI application
 index.html ← frontend SPA
 requirements.txt
 .env ← API keys (never commit)

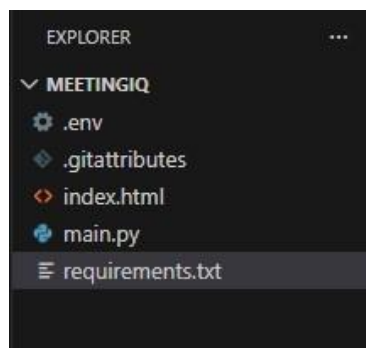


Figure 7: File Structure

5. Running the Application

Start the Uvicorn development server by running the following command from the project root directory with your virtual environment active:

```
python main.py
```

The terminal will display server startup messages from Uvicorn. Open <http://127.0.0.1:8000> in any modern web browser to access the application. You will be greeted by the upload interface immediately.

6. Development Environment (Recommended IDEs)

- Visual Studio Code: <https://code.visualstudio.com/> (recommended — excellent Python and HTML support, free)
- PyCharm Community Edition: <https://www.jetbrains.com/pycharm/download/> (excellent Python debugging tools, free)

7. Optional Reference Resources

- Groq API Documentation: <https://console.groq.com/docs>
- Google Gemini API Documentation: <https://ai.google.dev/gemini-api/docs>
- FastAPI Documentation: <https://fastapi.tiangolo.com/>
- MoviePy Documentation: <https://zulko.github.io/moviepy/>
- Whisper Model Card: <https://huggingface.co/openai/whisper-large-v3>
- python-dotenv Documentation: <https://pypi.org/project/python-dotenv/>

Project Flow

1. Environment Setup and Dependency Configuration

This phase focuses on preparing a fully isolated and correctly configured development environment for MeetingIQ, validating all foundational components, and ensuring the application can be started successfully before any feature development begins. A clean and reproducible environment at this stage prevents a large class of dependency conflicts and configuration errors later in development.

Activity 1.1: Python Environment and Package Installation

Create and activate a dedicated Python virtual environment named `meetingiq_env` to ensure complete dependency isolation from the system Python and from other projects. Install all required backend packages from the `requirements.txt` file using `pip`. Verify that Python 3.9 or higher is being used by running `python --version` in the activated environment. Confirm that all packages are importable without errors by starting a Python interpreter and importing each module individually. Test the Groq SDK specifically by confirming that the Groq client can be instantiated without an `ImportError`. Test the MoviePy installation by confirming that `AudioFileClip` can be imported from `moviepy.editor`. Verify that `google-generativeai` is importable and that `genai.GenerativeModel` can be instantiated.

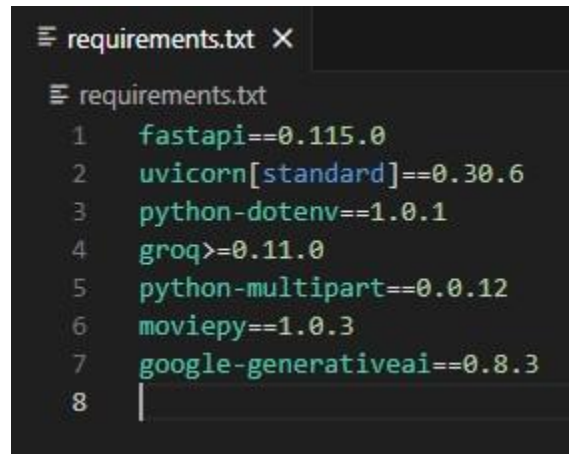
Activity 1.2: API Key Configuration and `.env` Setup

Create the `.env` file in the project root and populate it with valid `GROQ_API_KEY` and `GEMINI_API_KEY` values obtained from their respective developer consoles. Verify that `python-dotenv` correctly loads these variables by adding a temporary print statement to `main.py` that prints the first eight characters of each key at startup. Immediately add `.env` to the `.gitignore`

file. Confirm that the /health endpoint returns the expected JSON response listing the active model names, confirming that the backend has loaded the configuration correctly.

Activity 1.3: Project Structure and FastAPI Initialisation

Confirm that index.html is in the same directory as main.py. Launch the application using python main.py and verify that the Uvicorn server starts without errors and that the root URL (http://127.0.0.1:8000) returns the frontend HTML. Confirm that the /health endpoint returns a valid JSON response. Test the CORS configuration by opening the browser's developer console while on the frontend page and verifying there are no CORS-related errors in the network tab when the frontend attempts to reach the API.



```
requirements.txt X
requirements.txt
1 fastapi==0.115.0
2 uvicorn[standard]==0.30.6
3 python-dotenv==1.0.1
4 groq>=0.11.0
5 python-multipart==0.0.12
6 moviepy==1.0.3
7 google-generativeai==0.8.3
8 |
```

Figure 8: Dependencies Illustration

MILESTONE 1: Environment Setup Complete

All Python packages are installed and verified, API keys are securely configured and accessible, the FastAPI development server starts cleanly, and all three API endpoints return expected responses. The application is ready for feature development.

2. Backend API Development — FastAPI and AI Integration

This phase implements the complete server-side logic of MeetingIQ, including the audio extraction pipeline, the transcription routing logic, the intelligence extraction prompt and parsing, the dual-engine fallback architecture, and all supporting error handling and session management.

Activity 2.1: Intelligence Extraction System Prompt Design

The NOTES_SYSTEM_PROMPT is the most consequential configuration element in MeetingIQ. Design the complete twelve-field JSON schema in the prompt with realistic example values for all fields. Write the extraction rules section covering action item completeness (including implied items), priority definitions, deadline suggestion guidance, participant identification by name, and the critical final rule that only valid JSON must be returned with no markdown fences or explanatory text. Validate the prompt by testing it directly in the Groq playground or Gemini Studio before integrating it into the backend, using a sample meeting transcript to verify that the output matches the schema.

Activity 2.2: Audio Extraction Module Implementation

Implement the `extract_audio` function that accepts `input_path` and `output_path` as `Path` objects. Use a conditional check on the input file extension to route video formats to `VideoFileClip` and audio formats to `AudioFileClip` from `moviepy.editor`. Call `write_audiofile` on the resulting audio track with `bitrate 64k`, `verbose=False`, and `logger=None` to suppress `MoviePy`'s progress output. Wrap the function in a `try/except` block that raises a `RuntimeError` with a user-friendly message on failure. Add detailed debug logging throughout using `print` statements prefixed with `DEBUG` for easy filtering during development.

```

43 TRANSCRIPTION_MODEL = "whisper-large-v3"
44 NOTES_MODEL = "llama-3.3-70b-versatile"
45
46 SUPPORTED_EXTENSIONS = {
47     ".mp4", ".mkv", ".mov", ".avi", ".wmv", ".flv", ".webm",
48     ".m4v", ".3gp", ".ts", ".mts", ".m2ts",
49     ".mp3", ".wav", ".m4a", ".aac", ".ogg", ".flac", ".wma",
50     ".opus", ".webm", ".mpga", ".mpeg",
51 }
52
53 TEMP_DIR = Path(tempfile.gettempdir()) / "meetingiq"
54 TEMP_DIR.mkdir(exist_ok=True)

```

Figure 9: Extension Support

Activity 2.3: Transcription Module with Chunking

Implement the `transcribe_audio` function that checks the file size of the `audio_path` against the twenty-two megabyte threshold. For files below the threshold, call `client.audio.transcriptions.create` with the `whisper-large-v3` model, `response_format text`, `language en`, and `temperature 0.0`. Handle the case where the result may be either a string or an object with a `text` attribute. For files above the threshold, or files that fail the direct path with a size-related error, implement the chunked path using `AudioFileClip` with `subclip` extraction, sequential chunk file writing, independent transcription of each chunk, and joined result assembly.

Activity 2.4: Analysis Engine with Dual-Engine Fallback

Implement the `generate_notes_gemini` fallback function that constructs the prompt by combining the `NOTES_SYSTEM_PROMPT` with the transcript text, calls `gemini_model.generate_content`, strips any markdown code fences from the response, and returns the parsed JSON dictionary. Implement the `generate_notes` primary function that routes to `generate_notes_gemini` immediately for transcripts over forty thousand characters, otherwise calls the Groq LLaMA endpoint with `temperature 0.3` and `max_tokens 4000`, strips markdown fences from the response, parses the JSON, and falls back to `generate_notes_gemini` on any exception. This ensures that every processing request produces a structured result even under edge conditions.

Activity 2.5: Main Processing Endpoint Implementation (`/api/process`)

Implement the `process_recording` FastAPI route as an `async` function accepting a file `UploadFile` and a title form field. Validate the file extension against the `SUPPORTED_EXTENSIONS` set and return HTTP 400 for unsupported formats. Generate a UUID session ID and create a temporary session directory under the `TEMP_DIR` path. Write the uploaded file to disk, call `extract_audio`, call `transcribe_audio`, validate that the transcript is not empty, call `generate_notes`, inject the transcript and metadata into the notes dictionary, and

return the complete payload as a JSONResponse. Place the shutil.rmtree cleanup call in a finally block to ensure temp files are always deleted regardless of success or failure path.

MLESTONE 2: Backend API Development Complete

All three endpoints are operational, the audio extraction pipeline handles both video and audio inputs, the transcription module handles both direct and chunked paths, the analysis engine falls back to Gemini correctly, error handling is comprehensive on all routes, and the backend correctly validates inputs and returns structured JSON responses.

3. Frontend SPA Development

This phase builds the complete single-page application including the upload interface, the processing state management, the results dashboard, and all interactive elements. The frontend is the user-facing heart of MeetingIQ and must be carefully crafted to feel modern, intuitive, and professional.

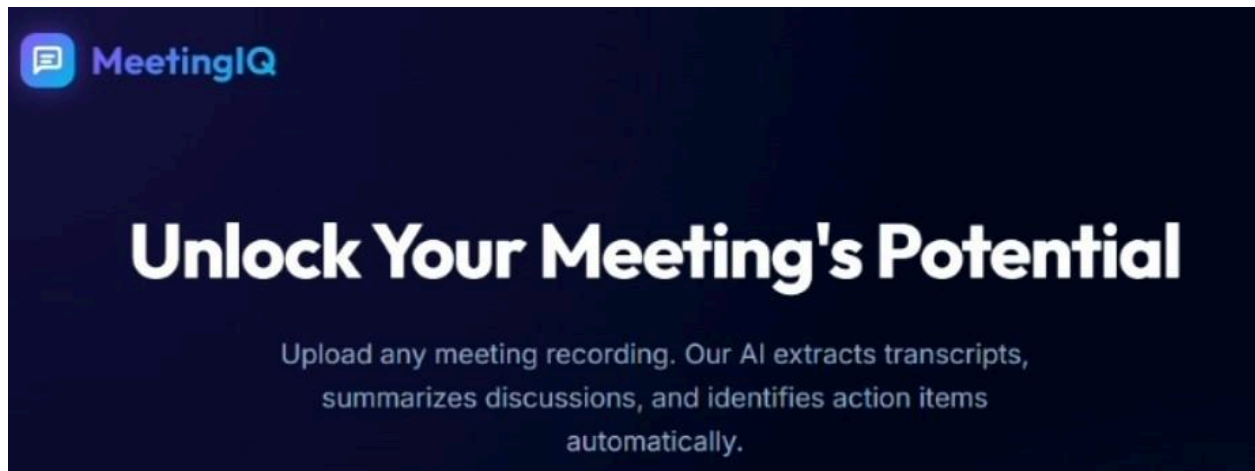


Figure 10: Frontend Illustration

Activity 3.1: Foundation — Design System and CSS Variables

Define the complete CSS custom property system covering the primary colour palette (purple #8b5cf6 with glow variant, secondary pink #ec4899, accent cyan #06b6d4), surface and background colours (deep navy #0f172a background, slate #1e293b surface, lighter slate #334155 surface-light), text colours at two levels of emphasis, glass and glass-border values for the glassmorphic treatment, and all semantic status colours (success green, warning amber, error red). Define the base radial gradient background on the body with two subtle corner glows in purple and cyan. Set up the CSS transitions for hover states on interactive elements and the animation keyframes for the spinner rotation and the results section fade-in.

Activity 3.2: Upload Section Implementation

Build the navigation bar with the gradient brand icon, the brand name with gradient text treatment, and appropriate responsive padding. Build the hero section with the primary headline and subtitle. Build the drag-and-drop zone with the dashed border, glassmorphic background, hover and dragover state transitions, the upload icon, instructional text, and the file name display area. Wire the dragover, dragleave, and drop events to the drop zone element and the change event to the hidden file input. Implement the handleFile function that captures the selected File object, displays the file name and size, and enables the Process Recording button.

Build the meeting title input field. Build the Process Recording button with the gradient background, gradient glow on hover, disabled state styling, and the upload icon.

Activity 3.3: Processing State Management

Build the full-screen loading overlay with backdrop filter blur, the spinning gradient ring animation, the loading status text element, and the processing subtext. Implement the processBtn on click handler that creates a FormData object, appends the file and title, sets the upload section to reduced opacity, shows the loading overlay, starts the status rotation interval that cycles through the four pipeline stage messages every five seconds, sends the multipart Fetch API request to /api/process, receives the JSON response, calls renderResults on success, and handles errors with an alert message. Clear the status interval in both the success and error paths and always hide the loading overlay in the finally block.



Figure 11: All Extension Support

Activity 3.4: Results Dashboard Implementation

Implement the renderResults function that populates all elements of the results section from the JSON response. Set the meeting title and processed-from subtitle in the dashboard header. Populate the three stats boxes with meeting effectiveness, sentiment, and action item count. Set the executive summary paragraph text. Implement the action items rendering that maps each action item to a card HTML string with a checkbox, the priority badge with the appropriate colour class, the category label, the task description, and the assignee and deadline metadata row. Implement the decision rendering that maps each decision to an accent-bordered card with the decision text and impact description. Implement the participant tags rendering. Set the next steps paragraph and the full transcript content. Show the results section, hide the upload section, and scroll to the top of the page.

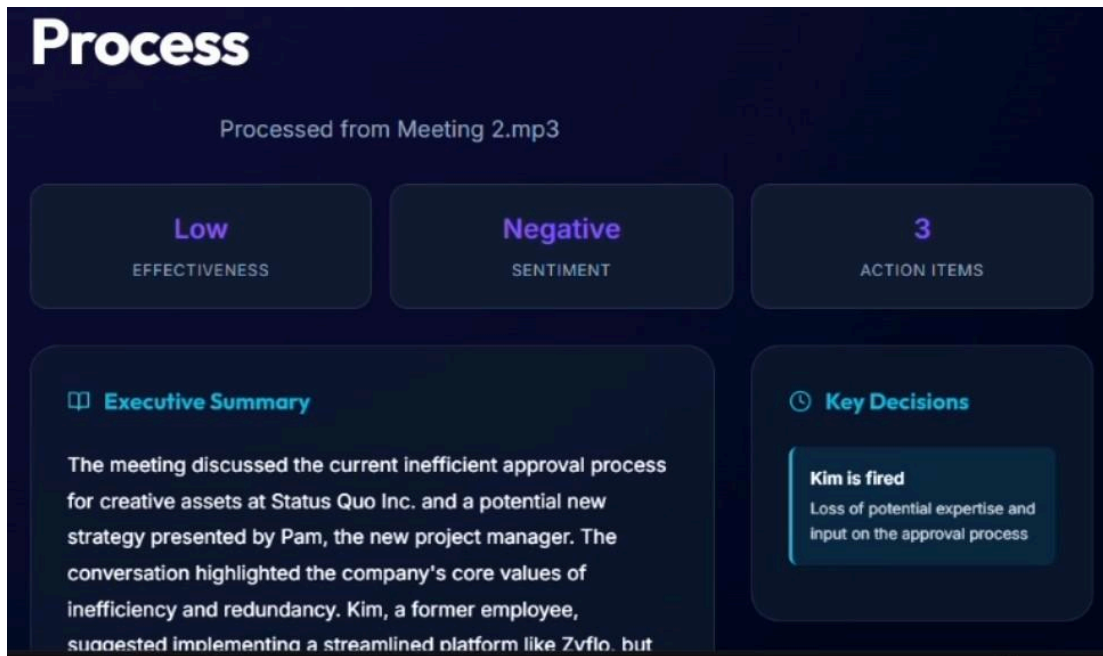


Figure 12: Analyzed Data

Activity 3.5: Reset Flow Implementation

Implement the resetUI function that hides the results section, shows and restores the upload section to full opacity, clears the selectedFile variable, resets the file input value, clears the file name display, disables the Process Recording button, and scrolls to the top of the page. Wire this function to the New Analysis back button in the results section header. Test the complete cycle of upload, process, view results, reset, and upload again to verify that no state leaks between sessions.



Figure 13: Transcription Illustration

MLESTONE 3: Frontend SPA Development Complete

The upload interface accepts files via drag-and-drop and file picker, the processing overlay provides clear status feedback, the results dashboard renders all twelve fields of the intelligence payload, interactive checkboxes function correctly, and the reset flow cleanly restores the upload interface. The application is visually polished and ready for user testing.

4. Testing, Optimisation, and Deployment

This final phase ensures that MeetingIQ is reliable, accurate, performant, and ready for real users. It covers functional testing of all features, validation of the chunked transcription pipeline, error handling verification, and deployment preparation.

Activity 4.1: Functional Testing and Pipeline Validation

Test the complete upload flow with a range of file formats including at minimum MP4, MOV, MP3, WAV, and M4A to verify that the audio extraction handles each correctly. Test with a short recording to verify the direct transcription path. Test with a recording that produces an extracted audio file over twenty-two megabytes to verify that the chunked path activates correctly and produces a continuous transcript. Test the meeting title field with and without a value to verify that the auto-detection path in the system prompt is exercised when no title is provided. Test the New Analysis flow multiple times in sequence to verify that the UI resets completely between sessions.

Activity 4.2: Intelligence Quality and Schema Validation

Test the analysis pipeline with a variety of meeting types including a standup with clear action assignments, a strategic discussion with multiple decisions, a brainstorming session with many implied next steps, and a status update meeting with few clear owners. In each case verify that the JSON response validates against the expected schema, that all twelve top-level fields are present, that action items include both explicit and implied tasks, that priority ratings are reasonable given the transcript content, and that deadline suggestions are grounded in language cues from the transcript rather than being generic placeholders.

MLESTONE 4: Testing, Optimisation, and Deployment Complete

Activity 4.3: Error Handling and Resilience Testing

Test the application's behaviour when an unsupported file format is uploaded and verify that the correct HTTP 400 response and user-facing error alert appear. Test the Groq API key invalid or missing case and verify that a user-friendly error message appears rather than a Python traceback. Test the Gemini API fallback by temporarily providing an invalid key and verifying that the analysis falls back correctly when the transcript is long enough to trigger Gemini routing. Test the frontend's handling of a stopped server by stopping the FastAPI process while the frontend is loaded and attempting to process a recording, verifying that the loading overlay is dismissed and a meaningful error alert is shown.

Activity 4.4: Deployment Preparation and Production Configuration

Replace the debug `uvicorn.run` call in `main.py` with a production-appropriate configuration using host `0.0.0.0` and the port specified by an environment variable defaulting to `8000`. Review all `HTTPException` detail strings to ensure no internal implementation details are exposed in production error responses. Ensure the `.env` file is listed in `.gitignore` and that no API keys are hard-coded anywhere in the codebase. Prepare deployment configuration for the target platform with environment variable injection via the platform's dashboard rather than a `.env` file. Conduct

one final end-to-end test covering the complete user journey: file selection, title entry, process click, status observation, results review, action item checking, transcript reading, and new analysis reset.

MeetingIQ is functionally complete, all file format paths work correctly, the chunked transcription pipeline handles large files, the dual-engine fallback is validated, error handling is comprehensive, and the application is configured for production deployment. MeetingIQ is ready to transform meeting recordings into actionable intelligence for real users.

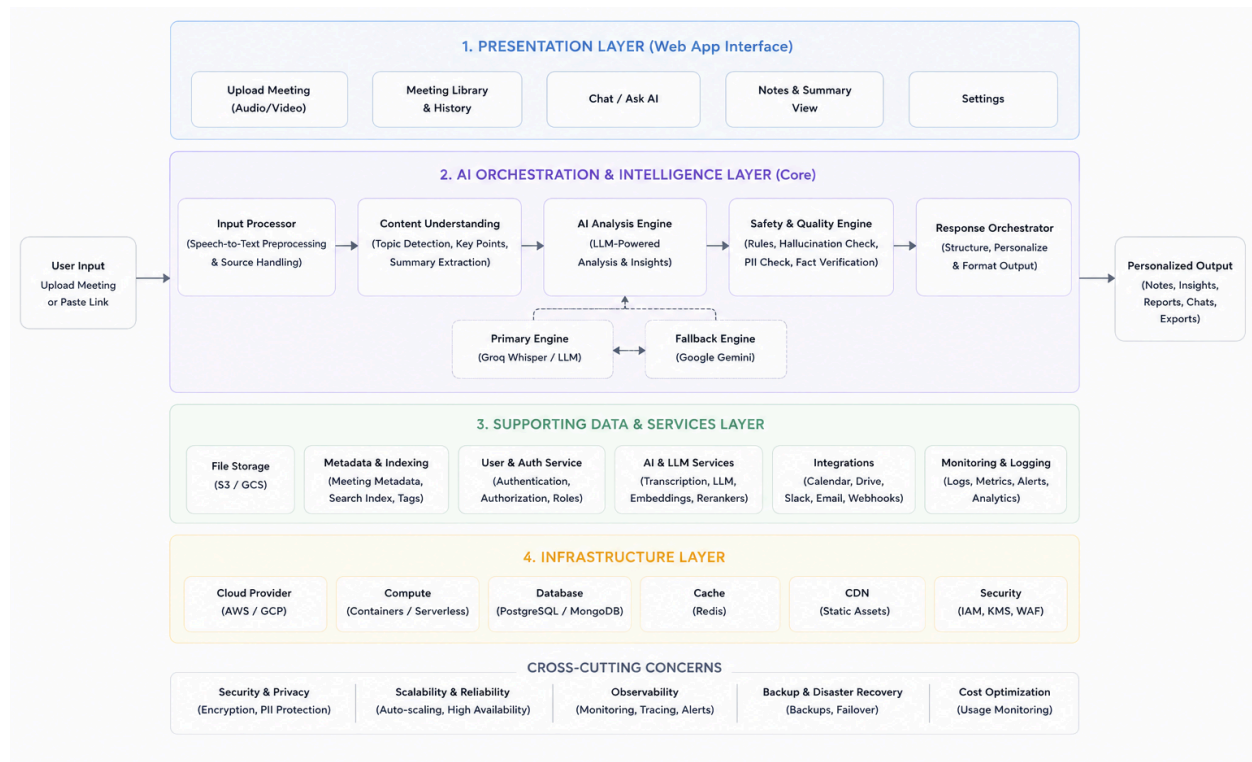


Figure 14: Testing & Deployment Illustration

Conclusion

MeetingIQ represents a thoughtful, purposeful, and technically sound approach to AI-powered meeting intelligence. By combining the transcription accuracy of Groq's Whisper large-v3 model with the structured extraction capability of LLaMA 3.3 70B as the primary engine and Gemini 1.5 Flash as an intelligent fallback, a carefully designed JSON schema and system prompt, robust audio processing through MoviePy, and an elegant glassmorphic frontend, the application delivers genuinely transformative value for professionals who need to extract structured intelligence from recorded conversations.

The technical architecture of MeetingIQ reflects a philosophy of purposeful simplicity. A lightweight FastAPI backend, a zero-framework frontend SPA, automatic session-scoped temporary storage, and a flat project structure make the application fast to develop, easy to deploy, and straightforward to maintain. The absence of a database, authentication system, or complex infrastructure means that MeetingIQ can be run by anyone with a Python environment and two API keys — lowering the barrier to entry while keeping the processing pipeline focused and reliable.

The dual-engine analysis architecture demonstrates that resilience and consistency can be achieved in AI-powered applications without sacrificing output quality. By ensuring that both the primary LLaMA engine and the Gemini fallback receive the same system prompt and produce the same JSON schema, MeetingIQ delivers a uniform user experience regardless of which backend path processed a given recording. The frontend never needs to know which engine was used, and users always see the same comprehensive intelligence dashboard.

Looking ahead, MeetingIQ has significant potential for future enhancement. Speaker diarisation to attribute transcript segments to individual participants, real-time streaming transcription for live meetings, integration with calendar systems for automatic recording retrieval, export to PDF or Word document formats, a searchable archive of processed meetings with full-text search across transcripts and intelligence reports, multilingual transcription and analysis support, and integration with project management platforms for direct action item synchronisation are all natural extensions of the current platform.

Ultimately, MeetingIQ demonstrates that responsibly integrated AI can provide immediate, measurable value in professional contexts. Meetings are expensive in time, attention, and coordination cost. The intelligence they generate is valuable. MeetingIQ ensures that none of that value is lost — transforming every recorded conversation into a structured, searchable, and immediately actionable intelligence report that captures exactly what was decided, who is responsible, and what comes next.